Week 5

Description to Motion

Jaerock Kwon, PhD
Associate Professor
Electrical and Computer Engineering
University of Michigan-Dearborn

Today is dense, and that is deliberate

Last Monday was 개천절 — no class. **Lab 4 is due today.**

This session is the densest of the foundation block:
URDF → TF2 → Gazebo → `ros2_control`, in one sitting.

Which is exactly why it was kept **live** rather than
sent home over the break.

- **Lab 5** — URDF, TF, and a deliberately broken map — due **10/19**
- **Track preference form** — due **Friday 10/16**
- **Teams announced next week**

1. URDF and Xacro

Links and joints

- **Links** — rigid bodies: visual, collision, inertial
- **Joints** — fixed, revolute, continuous, prismatic

Written literally, a URDF is repetitive and unmaintainable.

Xacro adds properties, macros, and expressions — expanded before anything reads it.

```
xacro urdf/mitt.urdf.xacro > /tmp/mitt.urdf
```

MRider's URDF has no literal dimensions

Every value comes from `config/mitt_dimensions.yaml`.

Whose header reads:

> ***NOTHING IN THIS FILE HAS BEEN MEASURED.***
> *Every value is an ESTIMATE derived from the vendor's published dimensions.*

Not an apology. A mechanism.

In November the Chassis track drops in measured numbers,
and no geometry anywhere else changes.

The trap being avoided

A predecessor's URDF carried:

- `chassis_mass = 300 kg` — for a ride-on car
- a controller config whose wheelbase **contradicted its own URDF**

Nobody noticed.

In simulation, nothing complains.

2. TF2

The tree, and who owns each edge

Transform	Published by	Kind
`map → odom`	`slam_toolbox`	dynamic — drift correction
`odom → base_link`	EKF	dynamic — local pose
`base_link → *`	`robot_state_publisher`	static, from the URDF

Ownership is exclusive, and enforced.

The Ackermann controller runs `enable_odom_tf: false` so the EKF is the **sole** publisher of `odom → base_link`.

Two publishers for one transform

The robot jitters.

- Nothing errors
- Both publishers are behaving correctly
- The tree looks complete in `view_frames`

A fourth silent failure, and one you can cause yourself.

Two practical facts

`base_link` is at the rear axle centre, on the ground plane (REP-105).

That is why `base_link_height` and `wheel_radius` are both `0.09`.
Not a coincidence — a derivation.

`tf2_echo` needs `use_sim_time:=true`.

Without it you compare wall clock against sim clock and get
extrapolation errors that look like a broken tree.

3. From description to motion

The chain

```
mitt.urdf.xacro
   | xacro expands
   ▼
/robot_description ──► robot_state_publisher ──► TF (base_link → *)
   |
   | ros_gz_sim create
   ▼
model spawned in Gazebo
   |  <gazebo><plugin gz_ros2_control>
   ▼
controller_manager   ── loads ──►  joint_state_broadcaster
   (inside the Gazebo process)      ackermann_steering_controller
```

Two load-bearing details

The controller manager runs inside the Gazebo process.

That is what `gz_ros2_control` does — and it is why the CycloneDDS problem is specifically about service **responses** crossing that boundary.

Actuation does not cross the ROS–Gazebo bridge.

Every entry in `gz_bridge.yaml` is `GZ_TO_ROS`: clock, scan, IMU, camera. Nothing in that file commands the vehicle.

The identical controller stack runs in simulation and on hardware, differing only in the `hardware_interface` plugin.

ADR-SW4. This is what the twin is **for**.

And that plugin does not exist

`mitt_controllers.yaml` is used **unchanged** in sim and on hardware.

Its header:

> **If a value here needs to be different on the real vehicle, that is a signal something is wrong with the model, not a reason to fork the file.**

`mitt_hardware` — the real plugin — is **not written yet.**

It is the Software track's keystone. **Merge 3, Week 13, exists to prove it.**

4. Ackermann kinematics

The bicycle model

$$\tan\delta = \frac{L}{R} \qquad R = \frac{L}{\tan\delta} \qquad \omega = \frac{v\tan\delta}{L}$$

Two consequences you have already felt:

- **$v = 0 \Longrightarrow \omega = 0$.** A car cannot rotate in place
- Steering is bounded, **so the turning radius is bounded**

The number that shapes this course

$$R_{min} = \frac{L}{\tan \delta_{max}} = \frac{0.63}{\tan 22.5^\circ} = 1.52 \text{ m}$$

Appears **twice** in `nav2_params.yaml` — `minimum_turning_radius` (planner) and `min_turning_radius` (controller).

And it is why `xy_goal_tolerance` is **0.6 m**, not 0.25 m.

A vehicle with a 1.52 m turning circle cannot make fine corrections near a goal. Ask it to and it **orbits** — which is exactly what happened during bring-up.

5. Silent failure #3

The worst kind

Week	Failure	Symptom
1	`ROS_DOMAIN_ID` mismatch	Nothing visible at all
3	QoS incompatibility	Connection never forms
4	**Wrong extrinsic**	**Everything runs, output quietly wrong**

The LiDAR publishes. TF is complete. SLAM runs. The map is wrong.

Every component is healthy — and faithfully computing the consequences of one bad number.

What that gate is defending

MRider's mapping acceptance gate:

> *Repeated observations of the same wall agree within **10 cm** over a **30 m** loop.*

Lab 5 has you introduce a **5 cm** error and watch it fail.

Now you know why the Chassis track measuring the real vehicle is not busywork.

Labs 4 and 5

Lab 5 — Describe the Robot

Add a rear camera to the URDF · verify TF · break an extrinsic by 5 cm · diagnose the map

Lab 4 — How Tightly Can It Turn?

Derive R_min · **predict before you measure** · find the clamp · explain a 2% gap

Both due 10/19.

The track preference form

Due **Friday 10/16**. Read Teams & Tracks first.

Track	For people who
Electrical & Firmware	like embedded work — or want to
Chassis & Measurement	like working with their hands
Simulation & Validation	like finding what everyone assumed was fine
Software & Autonomy	want the deepest ROS 2 work

Teams are **assigned**, announced 10/19. "No experience" is a fine answer.